

TECHNICAL DOCUMENTATION · V1

AI Ops Insurance Document Classifier

Architecture, ML pipeline design, data flows, and system internals for a fully self-contained insurance document classification system.

Python 3.14+

Streamlit

scikit-learn

TF-IDF

Tesseract OCR

pdfplumber

joblib

REPOSITORY	BRANCH	DATE	STATUS
eliteclown/aiops	claude/technical-docs-architecture-ae1f2m	June 2026	v1 — Initial Release

AIOps classifies insurance documents (PDF, Word, Excel, scanned images) into a two-level hierarchy — Category → Subcategory — using a user-trained ML model. No external APIs, no cloud, no data leaves the machine.

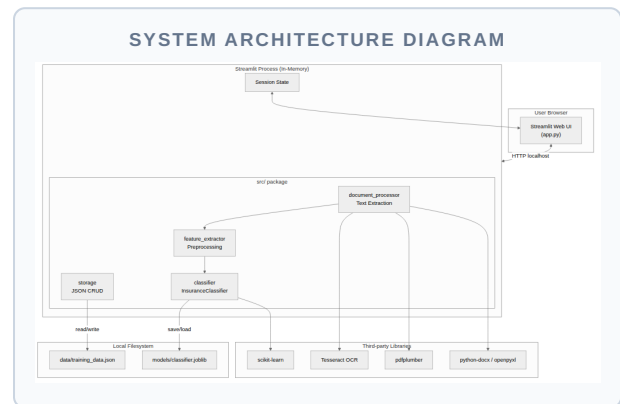
Capabilities

Capability	Detail
Ingest	PDF, DOCX, XLSX, PNG/JPG/TIFF/BMP/GIF
Extract	Native parsing for office formats; Tesseract OCR for images
Label	2-level: Category → Subcategory (e.g. Claims → Medical Records)
Train	4 scikit-learn algorithms, adaptive cross-validation
Predict	Top-5 ranked results with probability confidence
Persist	Training data → JSON; model → joblib

Technology Stack

Layer	Library
UI	Streamlit ≥ 1.35
ML	scikit-learn ≥ 1.4
PDF	pdfplumber ≥ 0.10
Office	python-docx · openpyxl · pandas
OCR	Pillow · pytesseract + Tesseract binary
Viz	matplotlib · seaborn

High-Level Architecture



Single-process Streamlit app — no separate backend, database server, or network calls. All state lives in-process (session state) or on local disk (JSON + joblib).

src/ Package

Module	Responsibility
app.py	UI orchestration — Train, Predict, Model Info pages
document_processor	Format-agnostic text extraction (all file types)
feature_extractor	Text normalization + keyword boosting (×3 TF weight)
classifier	InsuranceClassifier — pipeline, train/predict/save/load
storage	CRUD over data/ training_data.json
defaults	Built-in taxonomy: 6 categories, 6–8 subcategories each

Default Insurance Taxonomy

Category	Example Subcategories
Policy	Auto, Home/Property, Health, Life, Commercial
Claims	Claim Form, Medical Records, Adjuster Report
Underwriting	Application Form, Risk Assessment, Loss History
Financial	Premium Invoice, Payment Receipt, Audit Report
Legal	Endorsement, Exclusion Notice, Subrogation Letter
Correspondence	Cancellation Notice, Renewal Notice, Demand Letter

User-defined categories merge with defaults at runtime — no config needed.

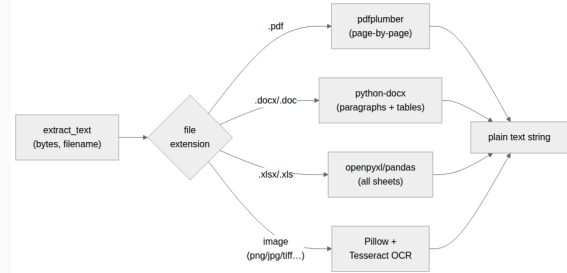
Feature Engineering

Keyword Boosting: User keywords are appended **3×**, inflating TF-IDF weight without a custom vectorizer — a lightweight human-in-the-loop signal.

```
processed = lowercase + strip_non_alphanum + collapse_ws
feature   = processed + " " + "
".join(keywords) * 3
```

Document Processing Pipeline

FORMAT DISPATCH — DOCUMENT_PROCESSOR.PY



PDF: pdfplumber, page-by-page text join.

DOCX: paragraphs + table cells joined with " | " .

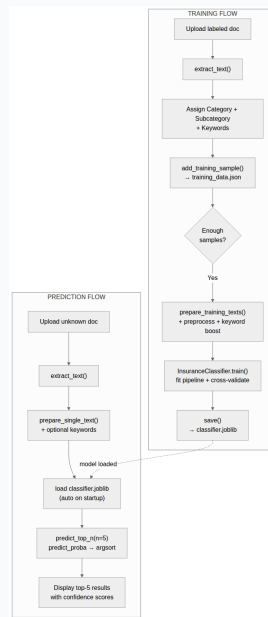
XLSX: all sheets linearized to col1 | col2 / val1 | val2 .

Images: Pillow opens file → Tesseract extracts text via OCR.

Directory Structure

```
aiops/
├── app.py                # Entry point & UI
├── requirements.txt
├── src/
│   ├── classifier.py     # ML engine
│   ├── defaults.py       # Insurance taxonomy
│   ├── document_processor.py
│   ├── feature_extractor.py
│   └── storage.py
├── data/                 # Auto-created
│   └── training_data.json
├── models/               # Auto-created
│   └── classifier.joblib
```

END-TO-END DATA FLOW — TRAINING (TOP) AND PREDICTION (BOTTOM)



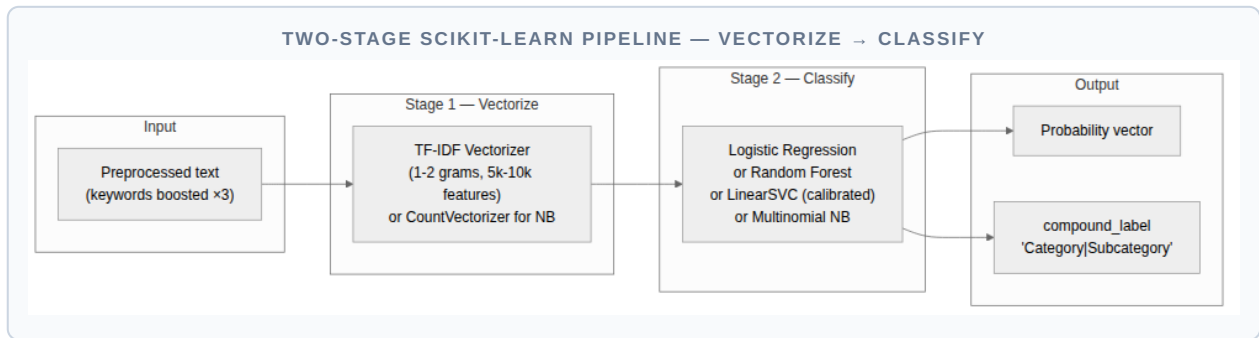
Training Flow

#	Step	Function
1	Upload + extract text	<code>extract_text(bytes, filename)</code>
2	Assign label + keywords	UI selection
3	Persist sample	<code>add_training_sample()</code> → JSON
4	Batch-prepare features	<code>prepare_training_texts()</code>
5	Fit pipeline	<code>InsuranceClassifier.train()</code>
6	Cross-validate	<code>cross_val_score()</code> adaptive folds
7	Serialize	<code>save()</code> → <code>classifier.joblib</code>

Prediction Flow

#	Step	Function
1	Auto-load model on startup	<code>InsuranceClassifier.load()</code>
2	Extract text from doc	<code>extract_text(bytes, filename)</code>
3	Preprocess + boost keywords	<code>prepare_single_text()</code>
4	Rank top-5 predictions	<code>predict_top_n(n=5)</code>
5	Parse compound label	<code>_parse_result()</code> splits on <code> </code>
6	Display confidence	Green ≥70% · Orange ≥40% · Red <40%

Compound labels: "Claims|Medical Records" is the single string the model learns. `_parse_result()` splits on `"|"` after prediction to recover both hierarchy levels.



Algorithm Comparison

Property	Logistic Regression	Random Forest	SVM Linear	Naive Bayes
Vectorizer	TF-IDF	TF-IDF	TF-IDF	CountVectorizer
Max features	10,000	5,000	10,000	10,000
N-grams	(1, 2)	(1, 2)	(1, 2)	(1, 2)
predict_proba	Native	Native	Platt scaling (CalibratedClassifierCV)	Native
Parallelism	—	n_jobs=-1 (all cores)	—	—
Best for	General baseline	Larger datasets	High-dim text	Very small datasets
Speed	Fast	Slower (200 trees)	Fast	Very fast

Cross-Validation Strategy

```

cv_folds = min(5, min_class_count)
# Only when min_class_count >= 2
cross_val_score(pipeline, texts, labels,
cv=cv_folds)
training_accuracy = cv_scores.mean()
  
```

Folds are capped so small classes never produce empty test splits.
Falls back to plain training accuracy when too few samples.

Model Serialization Schema

```

{
  "pipeline": Pipeline, # fitted
  "sklearn Pipeline": str,
  "classifier_name": str,
  "training_accuracy": float, # mean CV
  "or train acc": ndarray, # per-fold
  "cv_scores": scores | None,
  "classes_": list[str], # all
  "compound labels": int,
  "n_samples_trained": int
}
  
```

Training Sample (JSON Schema)

```
{
  "id": "uuid-v4",
  "filename": "claim_form.pdf",
  "text": "extracted full text...",
  "category": "Claims",
  "subcategory": "Medical Records",
  "keywords": ["medical", "claim
number"],
  "compound_label": "Claims|Medical Records",
  "timestamp": "2026-06-22T14:23:01"
}
```

Flat JSON array in `data/training_data.json`. Only `storage.py` reads or writes this file.

Three-Page UI

Page	Key Features
Train	Upload doc → preview → assign category/subcategory + keywords → add to dataset → select algorithm → train → see accuracy & CV scores
Test / Predict	Upload doc → optional keywords → classify → top-5 ranked results → confidence bar (green/orange/red)
Model Info	Metrics dashboard → class distribution chart → confusion matrix heatmap → training table → CSV export

Deployment

```
pip install -r requirements.txt
# Tesseract (optional, for image OCR):
# macOS: brew install tesseract
# Ubuntu: sudo apt-get install tesseract-ocr
streamlit run app.py # → http://localhost:8501
```

Session State Keys

Key	Type	Purpose
<code>classifier</code>	<code>InsuranceClassifier None</code>	Active trained model (auto-loaded on startup)
<code>train_text</code>	<code>str None</code>	Extracted text from training upload
<code>test_text</code>	<code>str None</code>	Extracted text for prediction

Error Handling

Scenario	Handling
Unsupported file type	<code>ValueError</code> shown as <code>st.error</code> type
Tesseract not installed	<code>RuntimeError</code> with install instructions
Fewer than 2 samples	Train button disabled with UI warning
Single-class dataset	Warning; training requires ≥ 2 distinct labels
Corrupt model file	<code>try/except</code> → returns <code>None</code> , sidebar warns

Extension Points

Extension	File to change
New ML algorithm	<code>classifier.py</code> — <code>_build_pipeline()</code> + <code>ALGORITHM_NAMES</code>
New file format	<code>document_processor.py</code> — <code>SUPPORTED_EXTENSIONS</code> + <code>extract_text()</code>
Database backend	<code>storage.py</code> only — public API is unchanged
REST API	Wrap <code>classifier.predict</code> + <code>extract_text</code> in FastAPI
LLM classifier	Implement same <code>train/predict/save/load</code> interface